Document Number: P1169R0
Date: 2018-10-07
Audience: EWG, LEWG
Reply-To: Barry Revzin, barry dot revzin at gmail dot com
Casey Carter, casey at carter dot net

static `operator()`

Contents

§ 1. Motivation

The standard libary has always accepted arbitrary function objects - whether to be unary or binary predicates, or perform arbitrary operations. Function objects with call operator templates in particular have a significant advantage today over using overload sets since you can just pass them into algorithms. This makes, for instance, `std::less<>{}` very useful.

As part of the Ranges work, more and more function objects are being added to the standard library - the set of Customization Point Objects (CPOs). These objects are Callable, but they don't, as a rule, have any members. They simply exist to do what Eric Niebler termed the "Std Swap Two-Step". Nevertheless, the call operators of all of these types are non-static member functions. Because *all* call operators have to be non-static member functions.

What this means is that if the call operator happens to not be inlined, an extra register must be used to pass in the `this` pointer to the object - even if there is no need for it whatsoever. Here is a simple example:

```
struct X {
    bool operator()(int) const;
    static bool f(int);
};

inline constexpr X x;

int count_x(std::vector<int> const& xs) {
    return std::count_if(xs.begin(), xs.end(),
#ifdef STATIC
    X::f
#else
    x
#endif
    );
}
```

`x` is a global function object that has no members that is intended to be passed into various algorithms. But in order to work in algorithms, it needs to have a call operator - which must be non-static. You can see the difference in the generated asm btween using the function object as intended and passing in an equivalent static member function:

Non-static call operator	Static member function

`count_x(std::vector<int, std::allocator<int> > const&):`
`push r12`
`push rbp`
`push rbx`
	`count_x(std::vector<int, std::allocator<int> > const&):`
	`push r12`
	`push rbp`
	`push rbx`

```
        sub     rsp, 16
        mov     r12, QWORD PTR [rdi+8]
        mov     rbx, QWORD PTR [rdi]
8       mov     BYTE PTR [rsp+15], 0
        cmp     r12, rbx
        je      .L5
        xor     ebp, ebp
.L4:
        mov     esi, DWORD PTR [rbx]
14      lea     rdi, [rsp+15]
        call    X::operator()(int) const
        cmp     al, 1
        sbb     rbp, -1
        add     rbx, 4
        cmp     r12, rbx
        jne     .L4
        add     rsp, 16
        mov     eax, ebp
        pop     rbx
        pop     rbp
        pop     r12
        ret
.L5:
        add     rsp, 16
        xor     eax, eax
        pop     rbx
        pop     rbp
        pop     r12
        ret
```

```
        mov     r12, QWORD PTR [rdi+8]
        mov     rbx, QWORD PTR [rdi]
        cmp     r12, rbx
        je      .L5
        xor     ebp, ebp
.L4:
        mov     edi, DWORD PTR [rbx]
        call    X::f(int)
        cmp     al, 1
        sbb     rbp, -1
        add     rbx, 4
        cmp     r12, rbx
        jne     .L4
        mov     eax, ebp
        pop     rbx
        pop     rbp
        pop     r12
        ret
.L5:
        pop     rbx
        xor     eax, eax
        pop     rbp
        pop     r12
        ret
```

Even in this simple example, you can see the extra zeroing out of `[rsp+15]`, the extra `lea` to move that zero-ed out area as the object parameter - which we know doesn't need to be used. This is wasteful, and seems to violate the fundamental philosophy that we don't pay for what we don't need.

The typical way to express the idea that we don't need an object parameter is to declare functions `static`. We just don't have that ability in this case.

§ 2. Proposal

The proposal is to just allow the ability to make the call operator a static member function, instead of requiring it to be a non-static member function. We have many years of experience with member-less function objects being useful. Let's remove the unnecessary object parameter overhead.

There are other operators that are currently required to be implemented as non-static member functions - all the unary operators, assignment, subscripting, and class member access. We do not believe that being able to declare any of these as static will have as much value, so we are not pursuing those at this time.

A common source of function objects whose call operators could be static but are not are lambdas without any capture, so this proposal also seeks to implicitly mark those static as well. This has the potential to break code which explicitly relies on the fact that the call operator is a non-static member function:

```
auto four = []{ return 4; };
auto p = &decltype(four)::operator();
(four.*p)(); // ok today, breaks with this proposal
```

The above code is pretty contrived, but a more direct example can be found in the deduction guide for `std::function` today which succeeds only if the call operator of the provided object is of the form `R(G::*)(A...)` (with optional trailing qualifiers). While this proposal will fix `std::function`, it would break user code that relies on custom deduction guides of the same style:

```
custom_function f = four; // ok today, f is a custom_function<int(void)>
                          // breaks with this proposal
```

Additionally, while there are many, many function objects in the standard library as it exists today that would benefit from this feature (`default_delete`, `owner_less`, the five arithmetic operations, the six comparisons, the the three logical operations, and the four bitwise operations), such a change would be an ABI break, so we are not pursuing that at this time. There is one, new function object that we could change for C++20: `identity`.

2.1. Language Wording

Change 7.5.5.1 [expr.prim.lambda.closure] paragraph 4:

> The function call operator or operator template is declared `static` if the *lambda-expression* has no *lambda-capture*, otherwise it is non-static. If it is a non-static member function, it is declared `const` ([class.mfct.non-static]) if and only if the *lambda-expression*'s *parameter-declaration-clause* is not followed by `mutable`. It is neither virtual nor declared `volatile`. Any *noexcept-specifier* specified on a *lambda-expression* applies to the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-declarator* appertains to the type of the corresponding function call operator or operator template. The function call operator or any given operator template specialization is a `constexpr` function if either the corresponding *lambda-expression*'s *parameter-declaration-clause* is followed by `constexpr`, or it satisfies the requirements for a `constexpr` function.

Simplify 7.5.5.1 [expr.prim.lambda.closure] paragraph 7:

> The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage having the same parameter and return types as the closure type's function call operator. The conversion is to "pointer to noexcept function" if the function call operator has a non-throwing exception specification. The value returned by this conversion function is the address of ~~a function~~ `F` ~~that, when invoked, has the same effect as invoking the closure type's function call operator. F is a~~ `constexpr` ~~function if the function call operator is a~~ `constexpr` ~~function~~ the function call operator. For a generic lambda with no *lambda-capture*, the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list, and the pointer to function has the same parameter types, as the function call operator template. The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization.

and 7.5.5.1 [expr.prim.lambda.closure] paragraph 9:

> The value returned by any given specialization of this conversion function template is the address of ~~a function~~ `F` ~~that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization.~~ `F` ~~is a~~ `constexpr` ~~function if the corresponding specialization is a~~ `constexpr` ~~function~~ the corresponding specialization of the function call operator template.

Change 11.5 [over.oper] paragraph 6:

> An operator function shall either be a ~~non-static~~ member function or be a non-member function that has at least one parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration. It is not possible to change the precedence, grouping, or number of operands of operators. The meaning of the operators `=`, (unary) `&`, and `,` (comma), predefined for each type, can be changed for specific class and enumeration types by defining operator functions that implement these operators. Operator functions are inherited in the same manner as other base class functions.

Change 11.5.4 [over.call] paragraph 1:

> `operator()` shall be a ~~non-static~~ member function with an arbitrary number of parameters. It can have default arguments. [...]

2.2. Library Wording

Change 19.14.10 [func.identity], `identity`

```
struct identity {
  template<class T>
    static constexpr T&& operator()(T&& x) ~~const~~;
```

```
  using is_transparent = unspecified;
};
template<class T>
  static constexpr T&& operator()(T&& x) ~~const~~;
```

1 Effects: Equivalent to `return std::forward<T>(t)`.

Change the deduction guide for `function` in 19.14.14.2.1 [func.wrap.func.con], paragraph 13:

```
template<class F> function(F) -> function<see below>;
```

Remarks: This deduction guide participates in overload resolution only if `&F::operator()` is well-formed when treated as an unevaluated operand. In that case, if `decltype(&F::operator())` is either of the form $\texttt{R(G::*)(A...)}\ cv\ \texttt{\&}_{opt}\ \texttt{noexcept}_{opt}$ for a class type `G` or of the form $\texttt{R(*)(A...)\ noexcept}_{opt}$, then the deduced type is `function<R(A...)>`.